

你好，20232311 朱峰正！今天我们来学习 **Linux命令**。

Linux系统

想象一下，你的电脑就像一个家，里面有各种硬件设备：CPU是大脑，内存是工作台，硬盘是仓库。**操作系统**就是这个家的**管家婆**和**服务生**。

管家婆 负责管理所有硬件资源，核心是管好三样东西：

- **文件**：家里所有的物品，比如文档、照片，都整理成文件。
- **虚存**：当工作台（内存）不够用时，会把暂时不用的东西临时存到仓库（硬盘）的特定区域，这就是虚拟内存。
- **进程**：家里同时进行的各项活动，比如你在听音乐的同时还在下载文件，每个活动就是一个进程。

服务生 则负责和你打交道，提供两种服务方式：

- **GUI**：就像家里的智能触控面板，点点鼠标就能控制家电，直观方便。
- **CLI**：就像用对讲机给管家下达精确的语音指令，高效、直接。
- **系统调用**：这是服务生和管家婆之间的内部通话，当你通过CLI或GUI下达指令后，服务生会通过系统调用来让管家婆真正执行。

从Unix到Linux：Unix是一个很早就出现的、非常稳定强大的操作系统，但早期它很昂贵且不开放。后来，一位名叫林纳斯·托瓦兹的大学生，为了能在家用电脑上体验Unix的强大功能，自己动手写了一个系统内核，这就是Linux的开始。他选择将Linux开源，意味着所有人都可以免费使用、学习和改进它，这吸引了全世界的开发者共同参与建设。

详解操作系统标准：

为了保证不同的软件能在不同的操作系统上顺利运行，就像不同品牌的电器都能插在标准的插座上一样，行业制定了一些标准。

- **POSIX标准**：可以理解为“**可移植操作系统接口**”标准。它定义了一套通用的API（应用程序编程接口），软件只要按照这个标准来开发，就能在不同的Unix-like系统（包括Linux）上编译和运行，大大提高了软件的可移植性。对于电信行业来说，这意味着为网络设备开发的监控、管理软件，可以更容易地部署在不同的系统上。
- **LSB标准**：可以理解为“**Linux标准库**”。它基于POSIX，但更具体地规定了Linux发行版应该包含哪些库文件、它们的存放位置、系统命令的路径等。目标是让同一个软件包能在遵守LSB标准的不同Linux发行版（比如Ubuntu、Red Hat）上直接安装运行，减少适配工作。这对电信运营商管理大量不同厂商的服务器非常有益。

Linux发行版

Linux本身只是一个系统内核，就像汽车只有发动机还不能开。**Linux发行版** 就是给这个发动机配上方向盘、轮胎、座椅和车身，组装成一辆完整的、可以直接上路的汽车。不同的组装厂（社区或公司）出于不同目的，组装出了不同特色的汽车。

重要发行版介绍：

- **Ubuntu**：非常流行的“家用轿车”，对新手友好，桌面体验好。 [官网](#)
- **Debian**：以稳定性著称的“经典车型”，Ubuntu等很多发行版是基于它开发的。 [官网](#)
- **Red Hat Enterprise Linux**：企业级的“豪华商务车”，提供付费的商业支持和服务，极其稳定可靠，广泛用于电信、金融等关键业务服务器。
- **Fedora**：Red Hat赞助的“概念车”或“测试平台”，包含最新技术，成熟后可能用于RHEL。
- **CentOS**：曾经是RHEL的“免费克隆版”，完全兼容RHEL但不含商业支持，非常适合搭建企业级服务器。现在由Red Hat管理，方向有所调整。
- **Kali Linux**：专为网络安全测试设计的“特种车辆”，预装了大量的安全工具。 [官网](#)

国产操作系统介绍：

出于信息安全和国家战略的考虑，我国也大力发展自己的操作系统，它们大多基于Linux内核。

- **openEuler**：面向数字基础设施的开源操作系统，由华为贡献，社区运营，特别适合服务器、云计算、边缘计算等场景。 [官网](#)
- **统信UOS**：基于Deepin，主打桌面办公和国产软硬件生态适配的商业发行版。
- **深度操作系统**：拥有美观易用桌面的Linux发行版，在个人用户中口碑很好。 [官网](#)
- **麒麟系列操作系统**：包括中标麒麟、银河麒麟等，主要面向国防、政务、关键行业领域。

Linux包管理工具：

在Linux上安装、卸载、更新软件，不像在Windows上那样需要去网站下载安装包。Linux发行版提供了**软件仓库**（一个巨大的、经过测试的软件库）和**包管理工具**（管理软件的工具），就像手机上的“应用商店”。

我们以Debian/Ubuntu系的 **apt** 为例，它的基本使用就像在应用商店里操作：

1. **升级系统**：`sudo apt update` (刷新软件仓库列表，看看有哪些更新) -> `sudo apt upgrade` (执行升级)
2. **查找应用**：`apt search 软件名` (在仓库里搜索含有关键词的软件包)
3. **安装应用**：`sudo apt install 软件包名` (从仓库下载并安装指定软件)
4. **卸载应用**：`sudo apt remove 软件包名` (卸载软件但保留配置文件) 或 `sudo apt purge 软件包名` (彻底卸载软件和配置文件)
5. **查看帮助**：`apt --help` 或 `man apt` (查看详细的使用手册)

其他包管理工具简介：

- **yum / dnf**：是Red Hat/CentOS/Fedora系使用的包管理工具，功能和apt类似，命令也相似，例如 `yum install 软件包名` 或 `dnf install 软件包名`。
- **Windows下的类似工具**：Windows后来也推出了类似的包管理工具，如 **WinGet**，可以通过命令行来搜索和安装软件，例如 `winget search 软件名`。

Linux系统安装

你好，朱峰正！今天我们来聊聊怎么把Linux系统“请”到你的电脑里。这就像给新家选个合适的管家，安装方式不同，体验也大不一样。

裸机安装

这是最“原汁原味”的方式，就像把原来的操作系统整个换掉，让Linux直接管理你的电脑硬件。

- **优点**：性能最好，能100%发挥硬件能力。
- **注意**：操作前 **一定要备份好重要数据**，因为这会清空你的硬盘。好在 **Linux对硬件要求很低**，很多老电脑都能流畅运行。

WSL安装

如果你主要用Windows，但又想偶尔体验一下Linux命令，WSL（Windows Subsystem for Linux）是**最简单方便**的选择。

- **它是什么**：可以理解为在Windows系统里开了一个“Linux虚拟机”，但深度集成，占用资源少，启动快。
- **适合谁**：初学者、开发者，想在Windows环境下使用Linux命令行工具。

VMWare安装Linux发行版

VMware是一个功能强大的虚拟化软件，它能在你的Windows或macOS系统上**模拟出一台完整的“虚拟电脑”**，然后你可以在里面安装任何Linux发行版。

- **优点**：**功能强大**，性能好，稳定。它有一个免费的个人使用版本叫VMware Workstation Player。
- **官网与下载**：你可以去 [VMware 官网](#) 查看并下载最新版的VMware Workstation Player。对于大多数想认真学习同学，我**优先推荐这种方式**，因为它既能让你拥有完整的Linux体验，又不会影响你原来的系统。

VirtualBox安装Linux发行版

VirtualBox是另一个流行的虚拟化软件，由甲骨文公司提供。

- **最大特点**：完全**开源**、免费，功能也很齐全。

- **官网：** [VirtualBox 官网](#)
- **和VMware比：** 对于基本学习来说，两者体验差不多。VirtualBox因为免费开源，受众非常广。

在派上安装（树莓派，飞腾派，香橙派）

这些“派”其实就是一种小巧、廉价的微型电脑主板。它们本身就是为运行Linux等开源系统设计的。

- **树莓派：** 最出名，社区资源极其丰富。
- **飞腾派/香橙派：** 国内推出的类似产品，性价比高。
- **怎么装：** 通常需要先把Linux系统的镜像文件写入一张SD卡，然后把SD卡插入派中通电启动即可。这是一个动手性很强的玩法，能让你深入了解硬件和系统的关系。

Docker安装

Docker是一种“容器”技术。你可以把它想象成一种 **超级轻量级的虚拟机**。

- **它不安装完整系统：** 它只运行一个隔离的“容器”，里面包含某个应用及其依赖环境。
- **适合场景：** 如果你想快速体验某个基于Linux的特定服务（比如一个网站服务器），而不需要完整的桌面环境，用Docker会非常快。
- **注意：** 这种方式不适合Linux初学者系统性地学习命令行和系统管理。

总结一下：

- **想彻底拥抱Linux -> 选 裸机安装**（务必备份！）
- **想在Windows下方便地使用Linux命令 -> 选 WSL**
- **想拥有最接近真实服务器的完整学习环境（推荐） -> 选 VMware 或 VirtualBox 创建虚拟机**
- **喜欢硬件DIY -> 可以买个 派 来玩**
- **只想快速运行某个服务 -> 可以了解 Docker**

请填写你安装的发行版名称:

请填写你安装的方式:

请填写你安装的硬件结构:

在VMware虚拟机中安装Kali Linux详细教程

朱峰正，今天我来手把手教你在VMware里安装Kali Linux系统。这个过程就像是在你的Windows电脑里“搭建”一个独立的Linux实验室，既安全又方便。

准备工作

在开始之前，我们需要准备好三样东西：

1. 下载并安装VMware Workstation

VMware是一个专业的虚拟化软件，它能让你在现有系统里运行多个独立的操作系统。

下载步骤：

- 访问 [VMware官网](#)
- 点击"下载"或"Products"，找到"VMware Workstation Player"
- 选择适合你Windows系统的版本下载（通常是Windows版本）
- **注意：** VMware Workstation Player对个人使用是免费的

2. 下载Kali Linux镜像文件

Kali Linux是专为网络安全测试设计的Linux发行版。

下载步骤：

- 访问 [Kali Linux官网](#)
- 选择"Kali Linux Virtual Machines"或直接下载ISO镜像
- 建议下载64位的ISO文件（文件后缀是.iso）
- 文件大小约3-4GB，下载需要一些时间

3. 检查电脑配置

确保你的电脑满足以下要求：

- **内存：** 至少8GB（分配给虚拟机4GB）
- **硬盘空间：** 至少20GB空闲空间
- **处理器：** 支持虚拟化技术（大多数现代CPU都支持）

安装步骤详解

第一步：创建新的虚拟机

1. 打开安装好的VMware Workstation Player
2. 点击"创建新虚拟机"
3. 选择"安装程序光盘镜像文件"，点击"浏览"找到你下载的Kali Linux ISO文件
4. VMware会自动检测到这是Kali Linux

第二步：配置虚拟机参数

这是最关键的一步，配置好坏直接影响使用体验：

- **操作系统类型**：选择"Linux"，版本选择"Debian 10.x 64位"
- **虚拟机名称**：可以命名为"Kali Linux"或你喜欢的名字
- **位置**：选择一个剩余空间较大的磁盘分区
- **磁盘容量**：建议分配 **40GB**，选择"将虚拟磁盘拆分成多个文件"
- **内存大小**：如果你的电脑有8GB内存，分配 **4GB** 给虚拟机；如果有16GB，可以分配 **8GB**

第三步：安装Kali Linux系统

1. 启动刚创建的虚拟机，会进入Kali Linux安装界面
2. 选择"Graphical install"（图形化安装）
3. **语言选择**：选择"中文（简体）"
4. **地区设置**：选择"中国"
5. **键盘布局**：选择"汉语"
6. **分区方式**：新手建议选择"使用整个磁盘"
7. **用户设置**：设置你的用户名、密码（**务必记住这个密码**）

第四步：系统安装过程

- 系统开始自动安装，这个过程需要 **15-30分钟**
- 安装完成后会提示重启
- **重要**：重启前记得"弹出"ISO镜像文件，否则会再次进入安装界面

安装后的基本设置

安装VMware Tools

这是 **提升使用体验的关键步骤**：

1. 启动Kali Linux系统
2. 在VMware菜单栏点击"虚拟机" → "安装VMware Tools"
3. 在Kali Linux桌面会出现VMware Tools的虚拟光盘
4. 打开终端，依次执行以下命令：

```
# 切换到root用户
sudo su

# 创建挂载点并挂载光盘
mkdir /mnt/cdrom
mount /dev/cdrom /mnt/cdrom

# 复制安装包并解压
cp /mnt/cdrom/VMwareTools-*.tar.gz /tmp/
cd /tmp
tar -xzf VMwareTools-*.tar.gz

# 运行安装脚本
cd vmware-tools-distrib/
./vmware-install.pl
```

1. 安装过程中所有提示都按回车选择默认选项
2. 安装完成后重启系统

VMware Tools的作用：

- **文件拖拽**：可以在主机和虚拟机之间直接拖拽文件
- **剪贴板共享**：复制粘贴内容在两个系统间通用
- **更好的显示效果**：自动适应窗口大小
- **性能优化**：系统运行更流畅

可能遇到的问题及解决方法

问题1：虚拟机无法启动

- **原因**：可能是CPU虚拟化功能未开启
- **解决**：重启电脑进入BIOS设置，找到"Virtualization Technology"选项并启用

问题2：安装过程卡住

- **原因**：可能是镜像文件损坏或下载不完整
- **解决**：重新下载Kali Linux镜像文件，验证MD5校验值

问题3：网络连接问题

- **原因**：虚拟机网络配置不当
- **解决**：在VMware设置中，网络适配器选择"NAT模式"

学习建议

安装完成后，我建议你这样开始学习：

1. **熟悉终端**：打开终端（快捷键Ctrl+Alt+T），尝试输入一些基本命令如 `ls` 、 `pwd` 、 `cd`
2. **更新系统**：执行 `sudo apt update && sudo apt upgrade` 保持系统最新
3. **探索工具**：Kali预装了很多安全工具，可以先从 `nmap` 、 `wireshark` 等基础工具了解起

记住：这个虚拟机环境是你的安全实验场，可以随意尝试各种命令，不用担心搞坏主机系统。

整个安装过程虽然步骤较多，但只要按顺序操作，基本上都能成功。如果在某个步骤遇到困难，可以随时查看VMware的官方文档或者Kali Linux的安装指南。

Linux系统 Shell

你好，朱峰正！今天我们来深入聊聊Linux的“翻译官”——Shell。想象一下，你（用户）想和电脑硬件（CPU、内存等）沟通，但你们说着不同的“语言”。Shell就是站在中间的那个“超级翻译官”，它把你的命令翻译成系统能懂的语言，再把系统的反馈翻译成你能看懂的结果。

各种Shell的不同和切换方法

Linux世界里有多种Shell，就像人类世界有中文、英文、法文等多种语言一样，它们各有特色：

- **Bash**：这是 **最流行、最常见** 的Shell，可以说是Linux世界的“普通话”。大多数Linux发行版的默认Shell就是它。功能强大，兼容性好，对初学者非常友好。
- **Zsh**：可以看作是Bash的“增强版”，拥有更强大的自动补全、主题美化等功能。很多资深开发者和追求效率的人喜欢用它。著名的 **Oh My Zsh** 框架就是基于它的。
- **Fish**：它的设计目标是“友好、交互性强”，语法提示非常智能，对新手学习命令很有帮助。但它的脚本语法与Bash不完全兼容。
- **Dash**：一个轻量、快速的Shell，通常被系统脚本（比如开机启动脚本）使用，以保证执行效率。但交互式功能比较弱。

如何查看和切换Shell？

1. **查看当前使用的Shell**：

```
echo $SHELL
\\`\\`
```

这条命令会显示你当前登录Shell的完整路径，比如 `\`/bin/bash\``。

2. **查看系统已安装的Shell**：


```
cat /etc/shells  
\\`\\`
```

这条命令会列出你的系统里所有可以使用的Shell。

```
zsh: corrupt history file /home/kali/.zsh_history  
(kali@20232311zfz)-[~]  
$ echo $SHELL  
/usr/bin/zsh  
  
(kali@20232311zfz)-[~]  
$ cat /etc/shells  
# /etc/shells: valid login shells  
/bin/sh  
/usr/bin/sh  
/bin/bash  
/usr/bin/bash  
/bin/rbash  
/usr/bin/rbash  
/usr/bin/dash  
/usr/bin/tmux  
/bin/zsh  
/usr/bin/zsh  
/usr/bin/pwsh  
/opt/microsoft/powershell/7/pwsh  
/usr/bin/screen
```

3. 临时切换Shell：

直接在命令行输入你想切换的Shell的名字即可，比如想试试Zsh，就输入 `zsh` 。这种切换只对当前这个终端窗口有效，关闭后就失效了。

1. 永久更改默认Shell （请谨慎操作）：

使用 `chsh` （change shell）命令，例如想把默认Shell改为Zsh：

```
chsh -s /bin/zsh  
\\`\\`
```

****注意**：** 执行后需要****重新登录****系统才会生效。在不确定的情况下，建议先用临时切换的方式体验。

Bash下的常用快捷键及应用例子

熟练使用快捷键能极大提升你在命令行下的操作效率。下面是一些最常用的Bash快捷键，每个都配了两个例子：

- **Ctrl + A / Home** ：快速移动到命令行的行首。
 - **例子1**：你输入了一条很长的命令 `find /home -name "*.txt" -exec ls -l {} \;`，发现开头打错了，想修改。不用按一堆左箭头，直接 `Ctrl + A`，光标就跳到了最前面的 `f` 处。
 - **例子2**：命令执行后，你想再次执行但需要加个 `sudo` 提权。按 `Ctrl + A`，然后输入 `sudo` 和空格即可。

- **Ctrl + E / End** : 快速移动到命令行的行尾。
 - **例子1** : 你输入 `cat /etc/passwd` 后, 想在命令末尾加上 `| grep bash` 来过滤结果。直接按 **Ctrl + E** , 光标就跳到了 `d` 后面, 接着输入即可。
 - **例子2** : 你从历史命令里调出了一条旧命令, 想在结尾添加一个参数 `-l` 。按 **Ctrl + E** 直接到最后添加。
- **Ctrl + U** : 删除从光标处到行首的所有内容。
 - **例子1** : 你输入了 `sudo apt-get install wrong-package` , 发现包名打错了, 想全部重输。按 **Ctrl + A** 到行首, 再按 **Ctrl + U** , 整行命令就被清空了。
 - **例子2** : 你输入 `python3 long_script.py --option1 value1 --option2` , 输到一半发现 `--option1` 的值给错了。把光标移到 `--option1` 之前, 按 **Ctrl + U** , 就删掉了前面部分, 保留了你刚输入还没写完的 `--option2` 。
- **Ctrl + K** : 删除从光标处到行尾的所有内容。
 - **例子1** : 你输入 `ls -l /a/very/long/path/that/is/wrong` , 输完路径发现后面部分打错了。把光标移到正确部分之后、错误部分之前, 按 **Ctrl + K** 删除后面所有错误内容。
 - **例子2** : 你输入 `echo "Hello, World!" > file.txt` , 临时改变主意不想输出到文件了。把光标移到 `>` 符号前, 按 **Ctrl + K** 删除重定向部分, 然后直接回车输出到屏幕。
- **Ctrl + W** : 删除光标前的一个单词 (由空格隔开)。
 - **例子1** : 你输入 `cp file1.txt /wrong/directory/` , 发现目录路径错了。把光标放在路径末尾, 连续按 **Ctrl + W** 可以逐个单词地删除错误的路径。
 - **例子2** : 你输入 `ssh user@hostname -p 2222` , 发现端口号输错了。把光标放在端口号后面, 按 **Ctrl + W** 就删除了 `2222` 。
- **Ctrl + R** : 反向搜索历史命令。这是最强大的快捷键之一!
 - **例子1** : 你前几天用过一条复杂的 `find` 命令来清理日志, 但记不清完整命令了。按 **Ctrl + R** , 然后输入关键词比如 `find` 或 `log` , Bash就会帮你从历史命令里模糊搜索并显示出来。
 - **例子2** : 你记得曾经编译过一个程序, 用了 `gcc` 命令和一些参数。按 **Ctrl + R** , 输入 `gcc` , 就能快速找到那条编译命令, 不用重新输入。

Linux 下可以举一反三的核心命令

掌握了Shell这个“翻译官”, 现在我们来看看怎么给它下指令。Linux命令虽然成千上万, 但它们的结构是有规律的, 掌握了规律, 学起来就事半功倍。

使用ls命令介绍命令的组成: 命令, 参数, 选项

我们以最常用的 `ls` (list的缩写, 意思是“列出”) 命令为例, 来拆解一个典型Linux命令的构成:

- **命令** : 就是你要执行的核心操作, 比如 `ls` 。它告诉系统: “我想看看当前目录里有什么东西”。

```

(kali@20232311zfz)-[~]
$ ls
20232311
20232311.c
20232311.exe
20232311_shellcode.c
20232311_shellcode.exe
20232311_shellcode_upx.exe
20232311_upx.exe
20232311zfz.c
20232311zfz.exe
20232311zfz_hyp.exe
20232311zfz_hyp_upx.exe
20232311zfz_upx.exe
2025-11-16-193508_2558x1360_scrot.png
all-2.0.tar.gz
dEkGdPAL.jpeg
Desktop
Documents
Downloads
fullscreen_screenshot.png
met-10encoded20232311.exe
met20232311.exe
met_encoded10_php_20232311.php
met-encoded20232311.exe
met-encoded-jar20232311.jar
metjar20232311.jar
metphp_20232311.php
Music
Nessus-10.10.1-debian10_amd64.deb
nessus.license
Pictures
Public
pwn1117
pwn20232311
pwn2311
pwn232311
shellcode_20232311.c
shellcode_20232311.exe
shellcode_upx_20232311.exe
'Steam_v3.0.0-rc.3_linux_x64.tgz'
Templates
Videos
vmttools
VMwareTools-10.3.26-22085142.tar.gz

```

- **选项**：通常以 - （短选项）或 -- （长选项）开头，用来 **修饰命令的行为**，告诉命令“你想以什么样的方式执行”。例如：

- `ls -l`： `-l` 是一个选项，意思是“用长格式显示详细信息”。

```

(kali@20232311zfz)-[~]
$ ls -l
总计 872840
drwxrwxr-x 2 kali kali 4096 11月 16日 19:47 20232311
-rw-rw-r-- 1 kali kali 1573 10月 21日 01:22 20232311.c
-rwxrwxr-x 1 kali kali 106056 10月 21日 01:23 20232311.exe
-rw-rw-r-- 1 kali kali 2708 10月 20日 12:07 20232311_shellcode.c
-rwxrwxr-x 1 kali kali 106589 10月 20日 12:07 20232311_shellcode.exe
-rwxrwxr-x 1 kali kali 59485 10月 20日 12:07 20232311_shellcode_upx.exe
-rwxrwxr-x 1 kali kali 58952 10月 21日 01:23 20232311_upx.exe
-rw-rw-r-- 1 kali kali 2711 10月 21日 01:37 20232311zfz.c
-rwxrwxr-x 1 kali kali 106568 10月 21日 01:37 20232311zfz.exe
-rwxr-xr-x 1 root root 122880 10月 21日 02:01 20232311zfz_hyp.exe
-rwxr-xr-x 1 kali kali 111104 10月 21日 02:01 20232311zfz_hyp_upx.exe
-rwxrwxr-x 1 kali kali 59464 10月 21日 01:37 20232311zfz_upx.exe
-rw-rw-r-- 1 kali kali 378696 11月 16日 19:35 2025-11-16-193508_2558x1360_scrot.png
-rwxrwxr-x 1 kali kali 700131916 11月 12日 07:52 all-2.0.tar.gz
-rw-r--r-- 1 root root 46413 11月 12日 08:37 dEkGdPAL.jpeg
drwxr-xr-x 2 kali kali 4096 3月 16日 03:29 Desktop
drwxr-xr-x 2 kali kali 4096 10月 20日 08:24 Documents
drwxr-xr-x 2 kali kali 4096 12月 8日 19:33 Downloads
-rw-rw-r-- 1 kali kali 358355 11月 16日 19:55 fullscreen_screenshot.png
-rw-rw-r-- 1 kali kali 73802 10月 20日 08:47 met-10encoded20232311.exe
-rw-rw-r-- 1 kali kali 73802 10月 20日 08:47 met20232311.exe
-rw-rw-r-- 1 kali kali 1406 10月 20日 08:52 met_encoded10_php_20232311.php
-rw-rw-r-- 1 kali kali 73802 10月 20日 08:48 met-encoded20232311.exe
-rw-rw-r-- 1 kali kali 5265 10月 20日 08:50 met-encoded-jar20232311.jar
-rw-rw-r-- 1 kali kali 5265 10月 20日 08:49 metjar20232311.jar
-rw-rw-r-- 1 kali kali 1116 10月 20日 08:52 metphp_20232311.php
drwxr-xr-x 2 kali kali 4096 10月 20日 08:24 Music
-rw-rw-r-- 1 kali kali 66827124 11月 12日 07:20 Nessus-10.10.1-debian10_amd64.deb
-rw-rw-r-- 1 kali kali 1460 11月 12日 07:33 nessus.license
drwxr-xr-x 2 kali kali 4096 11月 25日 01:14 Pictures
drwxr-xr-x 2 kali kali 4096 10月 20日 08:24 Public
-rw-rw-r-- 1 kali kali 7418 2016年 8月 19日 pwn1117
-rw-rw-r-- 1 kali kali 7418 2016年 8月 19日 pwn20232311
-rwxrwxr-x 1 kali kali 7418 11月 16日 19:29 pwn2311
-rw-rw-r-- 1 kali kali 0 11月 16日 19:41 pwn232311
-rw-rw-r-- 1 kali kali 1573 10月 20日 10:49 shellcode_20232311.c
-rwxrwxr-x 1 kali kali 106077 10月 20日 10:49 shellcode_20232311.exe
-rwxrwxr-x 1 kali kali 58973 10月 20日 10:49 shellcode_upx_20232311.exe
-rw-rw-r-- 1 kali kali 70810277 10月 20日 09:41 'Steam_v3.0.0-rc.3_linux_x64.tgz'
drwxr-xr-x 2 kali kali 4096 10月 20日 08:24 Templates
drwxr-xr-x 2 kali kali 4096 10月 20日 08:24 Videos
drwxr-xr-x 3 root root 4096 11月 3日 03:06 vmttools
-r--r--r-- 1 kali kali 53949998 2023年 7月 14日 VMwareTools-10.3.26-22085142.tar.gz

```

- `ls -a`： `-a` 是另一个选项，意思是“显示所有文件，包括隐藏文件”。
- 选项可以组合使用，比如 `ls -la` 就是同时使用 `-l` 和 `-a` 的效果。

- **参数**：通常是命令作用的对象，比如文件或目录的路径。它告诉命令“你对谁执行这个操作”。
 - `ls /home`： `/home` 就是一个参数，意思是“列出 `/home` 目录下的内容，而不是当前目录”。
 - `ls -l file.txt`：这里 `-l` 是选项，`file.txt` 是参数，意思是“以长格式显示 `file.txt` 这个文件的详细信息”。

```

(kali@20232311zfz)-[~]
$ ls -a
.
..
20232311
20232311.c
20232311.exe
20232311_shellcode.c
20232311_shellcode.exe
20232311_shellcode_upx.exe
20232311_upx.exe
20232311zfz.c
20232311zfz.exe
20232311zfz_hyp.exe
20232311zfz_hyp_upx.exe
20232311zfz_upx.exe
2025-11-16-193508_2558x1360_scrot.png
all-2.0.tar.gz
.bash_logout
.bashrc
.bashrc.original
.BurpSuite
.cache
.config
.dbus
dEkGdPAL.jpeg
Desktop
.dnrc
Documents
Downloads
.face
.face.icon
fullscreen_screenshot.png
.gnupg
.ICEauthority
.java
.local
.BurpSuite
met-10encoded20232311.exe
met20232311.exe
met_encoded10_php_20232311.php
met-encoded20232311.exe
met-encoded-jar20232311.jar
metjar20232311.jar
metphp_20232311.php
.mozilla
.ms4
Music
Nessus-10.10.1-debian10_amd64.deb
nessus.license
Pictures
.pki
.local
Public
pwn1117
pwn20232311
pwn2311
pwn232311
.selected_editor
.set
shellcode_20232311.c
shellcode_20232311.exe
shellcode_upx_20232311.exe
'Steam_v3.0.0-rc.3_linux_x64.tgz'
.sudo_as_admin_successful
Templates
Videos
.viminfo
vmttools
VMwareTools-10.3.26-22085142.tar.gz
.wine
.Xauthority
.xsession-errors
.xsession-errors.old
.zprofile
.zsh_history
.zshrc

```

```

(kali@20232311zfz)-[~]
$ ls /home
kali

```

```

(kali@20232311zfz)-[~]
$ ls -l 20232311.c
-rw-rw-r-- 1 kali kali 1573 10月 21日 01:22 20232311.c

```

总结一下公式： 命令 [选项] [参数]

- 中括号 [] 表示这部分是 **可选的**。
- 一个命令可以只有命令本身，也可以带上选项、参数，或者都带。

我是谁？： who

当多人同时使用一台Linux服务器，或者你开了多个终端窗口时，你可能想知道：“现在还有谁也在这台机器上？” `who` 命令就是用来回答这个问题的。

例子：

1. 基本用法：直接输入 `who`

```
$ who
zhufeng  tty1          2024-12-19 09:30
bob      pts/0          2024-12-19 10:15 (192.168.1.100)
\`\`\`
```

- * 输出显示：用户 `\`zhufeng\`` 在 `\`tty1\``（可能是本地显示器）登录，时间是今天早上9点半。
- * 用户 `\`bob\`` 通过 `\`pts/0\``（一个远程终端）从IP地址 `\`192.168.1.100\`` 登录。

2. 查看更详细的信息： `who -u`

```
$ who -u
zhufeng  tty1          2024-12-19 09:30 09:30          1000
bob      pts/0          2024-12-19 10:15 .              1001 (192.168.1.100)
\`\`\`
```

- * `\`-u\`` 选项显示了空闲时间（IDLE）和进程ID（PID）。`\`bob\`` 那里的 `\`. \`` 表示他刚有活动，没有空闲

3. 查看系统的启动时间： `who -b`

```
$ who -b
          system boot 2024-12-19 06:05
\`\`\`
```

- * 这个命令告诉你系统最后一次是什么时候启动的。

`man who` 的详细内容：

`man` 命令是Linux最强大的自学工具，我们马上会详细讲。现在你先知道，输入 `man who` 就会显示出关于 `who` 命令的完整官方说明书，里面会详细列出所有可用的选项（比如我们刚才用的 `-u` , `-b` ）以及它们的含义。养成遇到新命令就查 `man` 的习惯，是成为Linux高手的关键。

```
(kali㉿20232311zfz)-[~]
$ who
kali      seat0      2026-03-16 03:29 (:0)

(kali㉿20232311zfz)-[~]
$ who -u
kali      seat0      2026-03-16 03:29 ?      1626 (:0)

(kali㉿20232311zfz)-[~]
$ who -b
      系统引导      2026-03-16 03:24

(kali㉿20232311zfz)-[~]
$ man who
```

我从哪里来？： `pwd`

在Linux的目录“迷宫”里穿梭，很容易迷路。 `pwd` （Print Working Directory的缩写）命令就像是一个**GPS定位器**，它永远告诉你：“你当前所在位置的完整路径是什么”。

例子：

1. **基本用法**：直接输入 `pwd`

```
$ pwd
/home/zhufeng/Documents
\\`\\`\\`
```

* 这表示你当前在 `\`zhufeng\`` 用户的家目录下的 `\`Documents\`` 文件夹里。

2. **避免符号链接的物理路径**： `pwd -P`

假设 `/var/www` 是一个指向 `/home/zhufeng/website` 的符号链接（类似Windows的快捷方式）。

```
# 在 /var/www 目录下执行：
$ pwd
/var/www
$ pwd -P
/home/zhufeng/website
\\`\\`\\`
```

- * `\`pwd\`` 显示的是你“看起来”在的路径（符号链接本身）。
- * `\`pwd -P\``（Physical）显示的是你“实际”在的物理路径。

3. 逻辑路径（默认）：`pwd -L`

接上例，仍在 `/var/www` 目录下：

```
$ pwd -L
/var/www
\`\`\`
```

- * `\`-L\``（Logical）是默认行为，显示逻辑路径。

相对路径 vs 绝对路径

这是理解Linux目录结构的核心概念：

- **绝对路径**：像家庭地址一样，从根目录 `/` 开始写起，完整地描述一个文件或目录的位置。例如 `/home/zhufeng/Documents/report.pdf`。无论你在哪个目录，这个路径都指向同一个文件。
- **相对路径**：像相对位置描述，从你当前所在目录（工作目录）开始计算。它更简短。
 - `.` 代表当前目录。比如 `./script.sh` 表示当前目录下的 `script.sh`。
 - `..` 代表上级目录。比如你当前在 `/home/zhufeng/Documents`，那么 `../Downloads` 就指的是 `/home/zhufeng/Downloads`。
 - 直接写文件名或目录名，如 `file.txt`，也是相对路径，指当前目录下的 `file.txt`。

使用场景：在脚本里或确保路径绝对正确时，用 **绝对路径**。在日常操作中，为了简便，常用 **相对路径**。

我到哪里去？：`cd`

`cd`（Change Directory的缩写）是你在Linux文件系统里“走路”的命令。它让你可以切换到任何你有权限进入的目录。

- `cd .`：切换到当前目录。这看起来好像没动，但在某些脚本或复杂命令组合里可能有其用途。
- `cd ..`：切换到上级目录。这是 **最常用的导航操作之一**，用于“往回走一层”。
- `cd ~` 或直接 `cd`：快速回到你的 **家目录**（Home Directory）。比如你的家目录是 `/home/zhufeng`，这个命令会让你立刻跳回去，非常方便。
- `cd /`：切换到 **根目录**。这是整个文件系统的起点。
- `cd -`：在两个目录之间 **快速切换**。比如你先从 `/home` 目录 `cd /var/log` 到了日志目录，然后输入 `cd -`，你就会立刻回到 `/home` 目录。再输入一次 `cd -`，你又回到了 `/var/log`。像书签一样好用。

```
(kali㉿ 20232311zfq)-[~]
$ pwd -P
/home/kali

(kali㉿ 20232311zfq)-[~]
$ pwd -L
/home/kali

(kali㉿ 20232311zfq)-[~]
$ cd .

(kali㉿ 20232311zfq)-[~]
$ pwd
/home/kali

(kali㉿ 20232311zfq)-[~]
$ cd ..

(kali㉿ 20232311zfq)-[/home]
$ pwd
/home

(kali㉿ 20232311zfq)-[/home]
$ cd ~

(kali㉿ 20232311zfq)-[~]
$ pwd
/home/kali

(kali㉿ 20232311zfq)-[~]
$ cd /

(kali㉿ 20232311zfq)-[/]
$ pwd
/

(kali㉿ 20232311zfq)-[/]
$ cd -
/home/kali
```

环境变量 CDPATH

这是一个可以提升效率的高级技巧。 CDPATH 环境变量可以设置一组“快捷目录”。当你使用 `cd` 后面跟一个 **相对路径** 时，Shell不仅会在当前目录找，还会在 CDPATH 设置的目录列表里找。

例如： 你经常需要跳转到几个不同的项目目录，但它们不在同一个父目录下。

```
# 设置 CDPATH，多个路径用冒号隔开
export CDPATH=.:~/projects:/work/scripts
```

```
# 现在，无论你在哪个目录，都可以：
cd project-alpha
```

```
(kali@20232311zfz)-[~]
$ cd ~/Desktop

(kali@20232311zfz)-[~/Desktop]
$ ls
20260316

(kali@20232311zfz)-[~/Desktop]
$ cd 20260316

(kali@20232311zfz)-[~/Desktop/20260316]
$ cd ..

(kali@20232311zfz)-[~/Desktop]
$ cd ..

(kali@20232311zfz)-[~]
$ cd 20260316
bash: cd: 20260316: 没有那个文件或目录

(kali@20232311zfz)-[~]
$ export CDPATH=~/Desktop

(kali@20232311zfz)-[~]
$ cd 20260316
/home/kali/Desktop/20260316

(kali@20232311zfz)-[~/Desktop/20260316]
$ █
```

如果当前目录没有 `project-alpha`，Shell会依次在 `~`（家目录）、`/projects`、`/work/scripts` 里寻找 `project-alpha` 目录，如果找到就直接跳转。这相当于给你的 `cd` 命令增加了多个“搜索起点”。

自学神器：man

`man`（manual的缩写）是Linux系统自带的**最权威、最全面的命令说明书**。可以说，**学会使用 `man` 比死记硬背任何命令都重要**。它就是你随身携带的Linux专家。

`man man`：重点各个小节的介绍

输入 `man man`，你会看到 `man` 命令自身的说明书。其中最关键的是它把帮助文档分成了不同的**小节**，因为同一个名字可能代表不同的东西：

- **第1节：用户命令**：普通用户可执行命令的说明，如 `ls`，`cp`。
- **第2节：系统调用**：内核提供的函数，如 `open`，`read`。
- **第3节：库函数**：C语言库里的函数，如 `printf`，`fopen`。
- **第4节：特殊文件**：通常是 `/dev` 下的设备文件。
- **第5节：文件格式**：配置文件的格式说明，如 `/etc/passwd`。
- **第6节：游戏**。

- **第7节：杂项。**
- **第8节：系统管理命令：** 通常需要root权限才能执行的命令，如 `ifconfig` 。

用 `man 1 printf` ； `man 3 printf` 显示区别

`printf` 既是一个终端命令，也是一个C语言函数。

- `man 1 printf` ： 查看 **第1节** ，显示的是如何在命令行里使用 `printf` 命令来格式化输出文本。
- `man 3 printf` ： 查看 **第3节** ，显示的是C语言编程中 `printf` 函数的用法、参数和返回值。
- `man printf` ： 如果不指定节号，`man` 会从数字最小的节开始找，所以 `man printf` 默认显示的就是 `man 1 printf` 的内容。

`man -k` 与 `apropos` ： 当你记不住完整命令时

如果你想做某件事，但不知道用什么命令，可以用 `man -k` （或它的等价命令 `apropos` ）进行 **关键字搜索** 。

- **例子：** 想找到“排序”相关的命令或函数。

```
$ man -k sort
  qsort (3)          - sorts an array
  sort (1)           - sort lines of text files
  tsort (1)          - perform topological sort
  \\\`
```

```
(kali@20232311zfz)-[~]
$ man -k sort
alphasort (3)          - scan a directory for matching entries
apt-sortpkgs (1)       - Utility to sort package index files
bsearch (3)           - binary search of a sorted array
bunzip2 (1)           - a block-sorting file compressor, v1.0.8
bzip2 (1)             - a block-sorting file compressor, v1.0.8
comm (1)              - compare two sorted files line by line
ctwill-proofsor (1)    - translate CWEB to TeX with mini-indexes
ctwill-refsort (1)    - translate CWEB to TeX with mini-indexes
magicsort (1)         - Categorize files by their file(1) magic
qsort (3)             - sort an array
qsort_r (3)           - sort an array
rpmsort (8)           - Sort input by RPM Package Manager (RPM) versioning
sort (1)              - sort lines of text files
sorter (1)            - Sort files in an image into categories based on file type
tsort (1)             - perform topological sort
twill-refsort (1)     - translate to with mini-indexes
versionsort (3)       - scan a directory for matching entries
vfs_dirsort (8)       - Sort directory contents
XConsortium (7)       - X Consortium information
```

```
(kali㉿20232311zfz)-[~]
$ man -f od
od (1)                - dump files in octal and other formats

(kali㉿20232311zfz)-[~]
$ whatis od
od (1)                - dump files in octal and other formats
```

它会在所有手册页的**名称和简短描述**中搜索关键词 `\`sort\``，然后列出所有相关条目，并注明它们属于哪一类。

man -f 与 whatis ：快速了解命令是干什么的

如果你知道一个命令的名字，但忘了它的具体用途，`man -f`（或 `whatis`）可以 **显示该命令的简短描述**。

- **例子**：忘了 `od` 命令是干嘛的。

```
$ man -f od
od (1)                - dump files in octal and other formats
\\`\\`\\`
```

输出告诉你，`\`od\`` 命令是用来用八进制和其他格式“倾倒”（显示）文件内容的。`\`whatis od\`` 效果相同。

man 的其他选项简介

- `man -a 命令名`：显示所有匹配章节的手册页，依次显示。
- 在 `man` 的浏览界面中，按 `/` 后输入关键词可以 **搜索**；按 `n` 跳转到下一个匹配项，按 `N` 跳转到上一个匹配项；按 `q` 退出。

man 与 info ， --help 的区别和联系

- `man`：是 **传统、标准** 的文档格式，一页纸说明，内容精炼。
- `info`：是GNU项目推崇的文档格式，支持 **超链接**，结构更清晰，适合复杂命令（如 `gcc`）的详细学习。可以理解为带目录的电子书。很多命令既可以用 `man` 查看，也可以用 `info 命令名` 查看，通常 `info` 的内容更详细。
- `** --help`

Linux 下课上常用命令

每个命令至少给出三个例子，最后附上man手册的参考信息。

od - 以多种格式显示文件内容

od (octal dump) 命令用于以八进制、十六进制等格式显示文件内容，常用于查看二进制文件。

例子：

1. 查看文本文件的八进制格式

```
echo "hello" > test.txt
od test.txt
# 输出显示每个字符的八进制编码
```

1. 查看二进制文件的十六进制格式

```
od -x /bin/ls | head
# 以十六进制显示ls可执行文件的前几行
```

1. 显示ASCII字符和十六进制

```
od -c -x test.txt
# 同时显示字符形式和十六进制形式
```

```
(kali㉿20232311zfz)-[~]
$ echo "hello" > test.txt

(kali㉿20232311zfz)-[~]
$ od test.txt
00000000 062550 066154 005157
00000006

(kali㉿20232311zfz)-[~]
$ od -x /bin/ls | head
00000000 457f 464c 0102 0001 0000 0000 0000 0000
00000020 0003 003e 0001 0000 6760 0000 0000 0000
00000040 0040 0000 0000 0000 6428 0002 0000 0000
00000060 0000 0000 0040 0038 000e 0040 001e 001d
00000100 0006 0000 0004 0000 0040 0000 0000 0000
00000120 0040 0000 0000 0000 0040 0000 0000 0000
00000140 0310 0000 0000 0000 0310 0000 0000 0000
00000160 0008 0000 0000 0000 0003 0000 0004 0000
00000200 0394 0000 0000 0000 0394 0000 0000 0000
00000220 0394 0000 0000 0000 001c 0000 0000 0000

(kali㉿20232311zfz)-[~]
$ od -c -x test.txt
00000000  h   e   l   l   o   \n
          6568   6c6c   0a6f
00000006
```

man参考： `man od` 查看所有格式选项和详细用法

echo - 显示文本

`echo` 命令用于在终端输出文本，`-n` 和 `-e` 是常用选项。

例子：

1. 基本文本输出

```
echo "Hello World"
# 输出: Hello World
```

1. 不换行输出（-n选项）

```
echo -n "正在处理..."
echo "完成"
# 输出: 正在处理...完成（在同一行）
```

1. 启用转义字符（-e选项）

```
echo -e "第一行\n第二行\t制表符"
# 输出两行文字，第二行有制表符
```

```
(kali㉿ 20232311zfz)-[~]
$ echo "Hello World"
Hello World

(kali㉿ 20232311zfz)-[~]
$ echo -n "正在处理 ..."
echo "完成"
正在处理 ... 完成

(kali㉿ 20232311zfz)-[~]
$ echo -e "第一行\n第二行\t制表符"
第一行
第二行 制表符
```

man参考： `man echo` 查看转义字符列表和选项说明

more / less - 分页显示文件

两者都用于分页查看文件内容，`less` 功能更强大。

例子：

1. 查看长文件

```
more /var/log/syslog
# 按空格翻页，按q退出
```

1. 搜索内容

```
less /etc/passwd
# 输入"/root"搜索包含root的行
```

```
(kali㉿ 20232311zfz)-[~]
$ nano test.txt

(kali㉿ 20232311zfz)-[~]
$ less ~/test.txt
```

```
hello
asdfghjkl
zxcvbnm,
qwertyuiop
uhbygvtfdx
rootsdfghj
nihaoghjkl
hellovbnm,
nonononotfc
```

```
/root█
```

Session 动作 编辑 查看 帮助

```
hello
asdfghjkl
zxcvbnm,
qwertyuiop
uhbygvtfdx
rootasdfghj
nihaoghjkl
hellovbnm,
nonononotfc
```

```
~
~
~
~
```

1. 查看命令输出

dmesg | less

分页查看系统启动信息

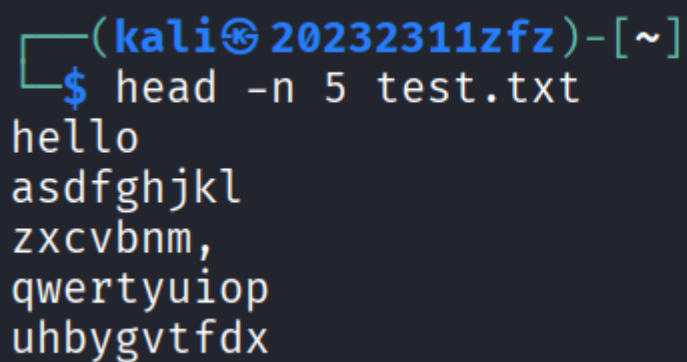
man参考： man more , man less 查看所有导航快捷键

head / tail - 显示文件开头/结尾

例子：

1. 查看文件前10行

```
head -n 10 /etc/passwd
```



```
(kali@20232311zfz)-[~]  
$ head -n 5 test.txt  
hello  
asdfghjkl  
zxcvbnm,  
qwertyuiop  
uhbygvtfdx
```

```
(kali㉿20232311zfz)-[~]
$ echo -e "banana\napple\ncherry" | sort
apple
banana
cherry

(kali㉿20232311zfz)-[~]
$ echo -e "100\n50\n200" | sort -n
50
100
200

(kali㉿20232311zfz)-[~]
$ echo -e "apple\nbanana\napple" | sort -u
apple
banana

(kali㉿20232311zfz)-[~]
$ echo "张三:25:北京" | cut -d: -f1,3
张三:北京

(kali㉿20232311zfz)-[~]
$ echo "abcdef" | cut -c2-4
bcd
```

1. 实时监控日志文件 (tail -f)

```
tail -f /var/log/auth.log
# 实时显示认证日志更新
```

1. 查看文件最后5行并持续监控

```
tail -n 5 -f /var/log/syslog
```

man参考: man head, man tail

sort - 排序文本

例子:

1. 按字母顺序排序

```
echo -e "banana\napple\ncherry" | sort
# 输出: apple banana cherry
```

1. 按数字大小排序

```
echo -e "100\n50\n200" | sort -n  
# 输出: 50 100 200
```

1. 去重排序

```
echo -e "apple\nbanana\napple" | sort -u  
# 输出: apple banana (去重)
```

man参考: `man sort`

cut - 截取文本列

例子:

1. 按字段截取

```
echo "张三:25:北京" | cut -d: -f1,3  
# 输出: 张三:北京 (以冒号分隔, 取第1、3字段)
```

1. 按字符位置截取

```
echo "abcdef" | cut -c2-4  
# 输出: bcd (第2到第4个字符)
```



```
(kali㉿20232311zfz)-[~]
$ echo -e "banana\napple\ncherry" | sort
apple
banana
cherry

(kali㉿20232311zfz)-[~]
$ echo -e "100\n50\n200" | sort -n
50
100
200

(kali㉿20232311zfz)-[~]
$ echo -e "apple\nbanana\napple" | sort -u
apple
banana

(kali㉿20232311zfz)-[~]
$ echo "张三:25:北京" | cut -d: -f1,3
张三:北京

(kali㉿20232311zfz)-[~]
$ echo "abcdef" | cut -c2-4
bcd
```

1. 处理系统文件

```
cut -d: -f1 /etc/passwd
# 提取所有用户名
```

man参考: `man cut`

cp - 复制文件/目录

例子:

1. 复制文件

```
cp file1.txt file2.txt
```

1. 递归复制目录

```
cp -r dir1 dir2
```

```
(kali㉿ 20232311zfz)-[~]
$ cd ~/Desktop

(kali㉿ 20232311zfz)-[~/Desktop]
$ ls
20260316

(kali㉿ 20232311zfz)-[~/Desktop]
$ cd ./20260316

(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ ls
316.txt

(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ cp 316.txt 316-1.txt

(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ ls
316-1.txt 316.txt

(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ cd ..

(kali㉿ 20232311zfz)-[~/Desktop]
$ cp -r 20260316 20260316-1

(kali㉿ 20232311zfz)-[~/Desktop]
$ ls
20260316 20260316-1

(kali㉿ 20232311zfz)-[~/Desktop]
$ cd ./20260316-1

(kali㉿ 20232311zfz)-[~/Desktop/20260316-1]
$ ls
316-1.txt 316.txt
```

1. 保留文件属性

```
cp -p important.conf backup/
```

man参考: `man cp`

rm - 删除文件/目录

例子:

1. 删除文件

```
rm tempfile.txt
```

1. 递归删除目录

```
rm -r old_directory
```

```
(kali㉿ 20232311z fz)-[~/Desktop/20260316-1]
$ ls
316-1.txt  316.txt

(kali㉿ 20232311z fz)-[~/Desktop/20260316-1]
$ rm 316.txt

(kali㉿ 20232311z fz)-[~/Desktop/20260316-1]
$ ls
316-1.txt

(kali㉿ 20232311z fz)-[~/Desktop/20260316-1]
$ cd ..

(kali㉿ 20232311z fz)-[~/Desktop]
$ rm -r 20260316-1

(kali㉿ 20232311z fz)-[~/Desktop]
$ ls
20260316
```

1. 强制删除（谨慎使用）

```
rm -f locked_file.txt
```

man参考： `man rm`

rmdir - 删除空目录

例子：

1. 删除空目录

```
rmdir empty_dir
```

1. 批量删除空目录

```
rmdir dir1 dir2 dir3
```

```
(kali㉿ 20232311zfz)-[~/Desktop]
$ mkdir 1

(kali㉿ 20232311zfz)-[~/Desktop]
$ mkdir 2

(kali㉿ 20232311zfz)-[~/Desktop]
$ ls
1  2  20260316

(kali㉿ 20232311zfz)-[~/Desktop]
$ rmdir 1 2

(kali㉿ 20232311zfz)-[~/Desktop]
$ ls
20260316
```

1. 删除多级空目录

```
rmdir -p path/to/empty/dir
```

man参考: `man rmdir`

file - 检测文件类型

例子:

1. 检测普通文件类型

```
file document.pdf
# 输出: PDF document, version 1.4
```

1. 检测脚本文件

```
file myscript.sh
# 输出: Bourne-Again shell script, ASCII text executable
```

1. 检测二进制文件

```
file /bin/ls
# 输出: ELF 64-bit LSB executable, x86-64, version 1...
```

```
(kali@20232311zfz)-[~/Desktop/20260316]
$ file 316.txt
316.txt: very short file (no magic)

(kali@20232311zfz)-[~/Desktop/20260316]
$ cd ~

(kali@20232311zfz)-[~]
$ file /bin/ls
/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=0243f2a3ad64d635299a574bbc8ef951dde9b21, for GNU/Linux 3.2.0, stripped

(kali@20232311zfz)-[~]
$ file pwn2311
pwn2311: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, for GNU/Linux 2.6.24, BuildID[sha1]=fb55ff390641d9430666f4c373725241894ef5a5, not stripped

(kali@20232311zfz)-[~]
$ file 20232311zfz.c
20232311zfz.c: C source, ASCII text
```

man参考： man file

free - 查看内存使用情况

例子：

1. 以人类可读格式显示

```
free -h
```

1. 持续监控内存

```
free -s 5
# 每5秒刷新一次
```

1. 显示详细统计

```
free -t
# 显示总计行
```

```
(kali㉿20232311zfz)-[~]
$ free -h
```

	total	used	free	shared	buff/cache	available
内存 :	3.8Gi	1.1Gi	2.1Gi	52Mi	889Mi	2.7Gi
交换 :	953Mi	0B	953Mi			

```
(kali㉿20232311zfz)-[~]
$ free -s 5
```

	total	used	free	shared	buff/cache	available
内存 :	4006676	1189380	2220644	53888	910776	2817296
交换 :	976556	0	976556			

	total	used	free	shared	buff/cache	available
内存 :	4006676	1195240	2214784	53892	910780	2811436
交换 :	976556	0	976556			

	total	used	free	shared	buff/cache	available
内存 :	4006676	1196492	2213524	53896	910792	2810184
交换 :	976556	0	976556			

	total	used	free	shared	buff/cache	available
内存 :	4006676	1198252	2211764	53900	910796	2808424
交换 :	976556	0	976556			

^C

```
(kali㉿20232311zfz)-[~]
$ free -t
```

	total	used	free	shared	buff/cache	available
内存 :	4006676	1190064	2219952	53904	910800	2816612
交换 :	976556	0	976556			
总量 :	4983232	1190064	3196508			

man参考: man free

du - 查看磁盘使用情况

例子:

1. 查看目录大小

```
du -sh /home/zhufeng
```

1. 查看子目录大小

```
du -h --max-depth=1 /var
```

```
(kali@20232311zfz)-[~]
$ du -sh ~/Desktop
16K    /home/kali/Desktop

(kali@20232311zfz)-[~]
$ du -h --max-depth=1 /home/kali
1.6M    /home/kali/.java
4.0K    /home/kali/Documents
382M    /home/kali/Downloads
5.1M    /home/kali/Pictures
1.7G    /home/kali/.cache
16K     /home/kali/Desktop
400K    /home/kali/.config
8.0K    /home/kali/.gnupg
4.0K    /home/kali/Videos
31M     /home/kali/.mozilla
42M     /home/kali/.local
11M     /home/kali/.msf4
76K     /home/kali/.pki
12K     /home/kali/.dbus
1.4G    /home/kali/.wine
4.0K    /home/kali/20232311
4.0K    /home/kali/Public
627M    /home/kali/.BurpSuite
4.0K    /home/kali/.set
206M    /home/kali/vmtools
4.0K    /home/kali/Templates
4.0K    /home/kali/Music
5.2G    /home/kali
```

1. 排除特定文件类型

```
du -sh --exclude="*.log" /var/log
```

man参考： `man du`

df - 查看文件系统磁盘空间

例子：

1. 人类可读格式

```
df -h
```

1. 查看特定文件系统

```
df -h /home
```

1. 显示inode信息

```
df -i
```

```
(kali㉿ 20232311zfz)-[~]
$ df -h
文件系统      大小  已用  可用  已用% 挂载点
udev          1.9G   0    1.9G   0% /dev
tmpfs         392M  1.3M  391M   1% /run
/dev/sda1      79G   38G   37G   51% /
tmpfs         2.0G   4.0K  2.0G   1% /dev/shm
tmpfs         1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
tmpfs         2.0G  748K  2.0G   1% /tmp
tmpfs         1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs         392M  124K  392M   1% /run/user/1000

(kali㉿ 20232311zfz)-[~]
$ df -h /home
文件系统      大小  已用  可用  已用% 挂载点
/dev/sda1      79G   38G   37G   51% /

(kali㉿ 20232311zfz)-[~]
$ df -i
文件系统      Inodes  已用 I   可用 I  已用 I% 挂载点
udev         483617   432  483185    1% /dev
tmpfs        500834   780  500054    1% /run
/dev/sda1    5251072 785404 4465668   15% /
tmpfs        500834     2  500832    1% /dev/shm
tmpfs        1024     1    1023    1% /run/credentials/systemd-journald.service
tmpfs       1048576   34  1048542    1% /tmp
tmpfs        1024     1    1023    1% /run/credentials/getty@tty1.service
tmpfs       100166   135  100031    1% /run/user/1000
```

man参考： `man df`

mv - 移动/重命名文件

例子：

1. 重命名文件

```
mv oldname.txt newname.txt
```

1. 移动文件到目录

```
mv file.txt /tmp/
```



```
(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ ls
316-1.txt  316.txt

(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ mv 316-1.txt 316-mv.txt

(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ ls
316-mv.txt 316.txt

(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ mv 316.txt ~/Desktop

(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ ls
316-mv.txt

(kali㉿ 20232311zfz)-[~/Desktop/20260316]
$ cd ..

(kali㉿ 20232311zfz)-[~/Desktop]
$ ls
20260316  316.txt
```

1. 批量移动

```
mv *.txt documents/
```

man参考: `man mv`

ping - 测试网络连接

例子:

1. 基本连通性测试

```
ping 8.8.8.8
```

1. 指定次数

```
ping -c 4 google.com
```

1. 设置时间间隔

```
ping -i 2 baidu.com
```

```
# 每2秒发送一次
```

```
(kali㉿ 20232311zfz)-[~]
$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=240 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=216 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=214 ms
^C
— 8.8.8.8 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2043ms
rtt min/avg/max/mdev = 214.281/223.506/239.994/11.686 ms

(kali㉿ 20232311zfz)-[~]
$ ping baidu.com
PING baidu.com (124.237.177.164) 56(84) bytes of data.
64 bytes from 124.237.177.164: icmp_seq=1 ttl=128 time=27.4 ms
64 bytes from 124.237.177.164: icmp_seq=2 ttl=128 time=33.0 ms
64 bytes from 124.237.177.164: icmp_seq=3 ttl=128 time=33.8 ms
^C
— baidu.com ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2054ms
rtt min/avg/max/mdev = 27.403/31.406/33.833/2.851 ms

(kali㉿ 20232311zfz)-[~]
$ ping -c 4 baidu.com
PING baidu.com (110.242.74.102) 56(84) bytes of data.
64 bytes from 110.242.74.102: icmp_seq=1 ttl=128 time=13.1 ms
64 bytes from 110.242.74.102: icmp_seq=2 ttl=128 time=30.6 ms
64 bytes from 110.242.74.102: icmp_seq=3 ttl=128 time=12.0 ms
64 bytes from 110.242.74.102: icmp_seq=4 ttl=128 time=13.3 ms

— baidu.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3086ms
rtt min/avg/max/mdev = 11.997/17.254/30.629/7.737 ms
```

man参考: `man ping`

history - 查看命令历史

例子:

1. 查看最近命令

```
history
```

```
(kali@20232311zfz)-[~]
$ history
 1 echo $SHELL
 2 ls
 3 ls -l
 4 ls -a
 5 ls /home
 6 ls -l 20232311.c
 7 who
 8 who -u
 9 who -b
10 man who
11 pwd
12 pwd -p
13 pwd -P
14 pwd -L
15 cd .
16 pwd
17 cd ..
18 pwd
19 cd ~
20 pwd
21 cd /
22 pwd
23 cd -
24 cd-
25 cd -
26 echo CDPATH
27 echo $CDPATH
28 env
29 ls
30 cd /Desktop
31 cd ~/Desktop
32 ls
33 cd ..
34 cd 316.txt
35 cd 20232311.c
36 cd ~/Desktop
37 ls
```

1. 查看最后10条命令

```
history 10
```

1. 搜索特定命令

```
history | grep ssh
```

```
118 ping 8.8.8.8
119 ping baidu.com
120 ping -c 4 baidu.com
121 history
```

```
(kali㉿20232311zfz)-[~]
$ history 10
113 mv 316.txt ~/Desktop
114 ls
115 cd ..
116 ls
117 cd ..
118 ping 8.8.8.8
119 ping baidu.com
120 ping -c 4 baidu.com
121 history
122 history 10
```

```
(kali㉿20232311zfz)-[~]
$ history | grep ping
118 ping 8.8.8.8
119 ping baidu.com
120 ping -c 4 baidu.com
123 history | grep ping
```

man参考： man history

学习建议： 对于每个命令，建议先用 `--help` 查看简要帮助，再用 `man` 命令深入学习详细用法。多动手实践是掌握这些命令的最佳方式。

Linux 管道

你好，朱峰正！今天我们来学习Linux中最强大的功能之一——管道。这就像是在命令之间搭建"流水线"，让数据能够自动传递和处理。

标准输入，标准输出，标准错误

在Linux中，每个程序运行时都会自动打开三个"数据流通道"：

- **标准输入：** 这是程序 **接收数据** 的通道，文件描述符编号为 **0**。默认情况下，它连接到你的键盘。当你在终端输入命令时，字符就是通过标准输入传递给程序的。

- **标准输出**：这是程序 **输出正常结果** 的通道，文件描述符编号为 **1**。默认显示在你的终端屏幕上。比如 `ls` 命令列出的文件列表，就是通过标准输出显示的。
- **标准错误**：这是程序 **输出错误信息和警告** 的通道，文件描述符编号为 **2**。默认也显示在终端屏幕上，但通常用不同的颜色区分。

简单理解：可以把程序想象成一个加工厂：

- 原材料从"标准输入"通道进入
- 加工好的产品从"标准输出"通道送出
- 生产过程中的故障报警从"标准错误"通道送出

输入输出重定向

Shell允许你改变这些默认的"数据流方向"，这就是重定向：

- **输出重定向**：用 `>` 或 `>>`
 - 命令 `>` 文件：将命令的标准输出 **覆盖** 写入到指定文件
 - 命令 `>>` 文件：将命令的标准输出 **追加** 到指定文件末尾
 - 命令 `2>` 错误文件：将标准错误重定向到指定文件
- **输入重定向**：用 `<`
 - 命令 `<` 文件：让命令从文件读取输入，而不是从键盘
- **管道**：用 `|`
 - 命令1 `|` 命令2：将命令1的 **标准输出** 作为命令2的 **标准输入**

学习命令行不仅仅是学习一堆命令，更是学习一种解决问题的思维方式。

当你遇到一个问题时，一个精通命令行的程序员会思考：

"我能否用几个现有的小工具，像搭积木一样把它们组合起来解决这个问题？"

不要仅仅把 Shell 当作一个启动程序的启动器，而要把它看作一个强大的编程环境和粘合剂。

1. 命令即是函数：

每个命令行工具（如 `grep`，`find`，`sed`，`awk`，`sort` 等）都可以被看作是执行单一功能的小型函数。它们遵循 Unix 哲学："只做一件事，并做到最好"。

2. 管道 (`|`) 是组合器：

Shell 的管道操作符允许你将这些小函数的输入和输出连接起来，创造出复杂的数据流处理管线。这是一种强大无比的组合式开发。

电信行业管道应用示例

以下是专门为电信行业设计的5个实用管道示例：

示例1：实时监控网络设备日志中的异常连接

```
tail -f /var/log/router/auth.log | grep -E "(FAILED|ERROR|unauthorized)" | awk '{print $1, $4, $5, $6}'
```

电信应用：实时监控路由器认证日志，提取失败登录尝试和未授权访问，用于安全审计和入侵检测。

示例2：分析基站信号质量统计数据

```
cat base_station_metrics.csv | grep -v "^#" | cut -d, -f2,5,7 | awk -F, '$3 < -90 {print "弱信号", $2, $5, $7}'
```

电信应用：处理基站性能数据，识别信号强度低于-90dBm的弱覆盖基站，按信号强度排序输出。

示例3：批量处理用户投诉数据并生成报告

```
find /telecom/complaints/ -name "*.log" -mtime -7 | xargs cat | grep "网络中断" | wc -l | awk '{print "一周内网络中断投诉次数: " $1}'
```

电信应用：统计过去一周内所有投诉日志中"网络中断"相关投诉的数量，用于服务质量监控。

示例4：监控网络流量异常峰值

```
ifconfig eth0 | grep "RX packets" | awk '{print $3, $5}' | sed 's/[:]/ /g' | awk '{print "接收包数量: " $1, "错误包比例: " $2}'
```

电信应用：实时提取网络接口统计信息，监控接收包数量和错误包比例，用于网络质量分析。

示例5：处理CDR话单数据并分析通话模式

```
cat call_detail_records.csv | awk -F, '{duration = $5; if(duration > 300) print $2, "长时间通话", duration}'
```

电信应用：分析通话详单，找出通话时长超过5分钟的前10个最长通话，用于业务模式分析或异常检测。

管道思维的核心优势：

- **模块化：**每个命令只负责一个简单任务
- **可组合性：**像搭积木一样自由组合
- **流式处理：**数据流动处理，内存占用小
- **可读性：**管道命令清晰表达了数据处理流程

在电信这种数据密集型的行业，掌握管道技术能让你高效处理海量的日志、监控数据和业务记录。

在 [MarkdownFlow 体验台](#) 制作

分享ID: 59c7cd94c1c50bc0a3a3b12c9d87a10